Representation and Tractability in Set Theoretic Estimation: Sheaves, Rectangles, and the Coupling Obstruction

Project `thejeb`

July 21, 2026

Abstract

The feasible set of a set theoretic estimation (STE) problem [2] lives in $\mathcal{P}(\Xi)$, and when the hypothesis space is a product over N variables, $|\Xi|$ is exponential in N: representing the feasible set extensionally hits a 2^N wall that is machine-checked in this project (`Ste.DynamicFrame.Model.card_allExactState`). This note asks when the feasible set can be represented *locally* and glued, and answers three pieces of the question with Lean-verified results in the module `Ste.Sheaf`. (i) *Gluing on the constraint side*: the assignment $J \mapsto \mathrm{feas}(J)$ from constraint subsets to partial feasible sets sends unions to intersections (`partialFeasibilitySet_iUnion`), so any cover of the constraint index recovers the global feasible set as the limit of local ones (`feasibilitySet_eq_iInter_cover`); on this side there is no obstruction. (ii) *A tractable upper bound*: if every constraint is variable-separable (a rectangle), the feasible set is itself a rectangle (`rectangular_feasibilitySet`) representable by one subset per variable—$\sum$-space, not $\prod$-space—with product cardinality (`rectangular_encard`). (iii) *The obstruction*: a single coupling constraint, the diagonal on two bits, is provably not a rectangle (`diagonal_not_rectangular`). This is the elementary form of the Abramsky–Brandenburger contextuality obstruction [1], and the same shape as this project's non-transitivity countermodel (`Ste.DynamicFrame.Counterexample`). Four further Lean modules push toward decomposition: `Ste.Decomposition` proves that disconnected constraint blocks give a product feasible set with multiplicative cost (`feasibilitySet_blockFamily`, `encard_feasibilitySet_blockFamily`); `Ste.Support` gives constraints a scope and shows the diagonal's scope is exactly $\{0,1\}$ (`diagonal_support_full`); `Ste.Conditioning` mechanizes variable elimination—conditioning commutes with feasibility, shrinks scopes, preserves rectangles, and, applied to a *cut* variable, disconnects the problem into independent blocks (`piecewise_mem_condition_feasib`) and `Ste.Treewidth` proves the general cut-elimination theorem (`cut_elimination`), the join-scope accounting behind bucket elimination, and the quantitative *bag bound*: the table of a constraint on a scope σ has at most $\prod_{v\in\sigma}|A_v| \le k^{w+1}$ rows (`table_encard_le`, `elimination_step`). A clearly-labelled outlook sketches the unproven remainder: multi-step elimination orders and junction trees, k-consistency as higher gluing, $\check{H}^1$ as the measure of intractability, and a treewidth-style tractability dichotomy [3, 4].

1 The problem: the 2^N wall

Combettes' set theoretic estimation [2] specifies a solution space Ξ, an index set I of pieces of information, and a property set $S_i \subseteq \Xi$ for each $i \in I$; the feasible (solution) set is $S = \bigcap_i S_i$. The framework is silent about *representation*: it treats S as a set, not as a data structure.

The silence becomes a cost the moment Ξ is a product. In the estimation problems this project cares about—dynamic frames over documents, constraint grammars, hyperframe lattices—the hypothesis space factors as $\Xi = \prod_{v\in V} A_v$ over a family of variables (claims, mentions, cells), and $|\Xi| = \prod_v |A_v|$ is exponential in $|V|$. The finite solver in `Ste.DynamicFrame.FiniteSolver` makes

the wall precise and machine-checked: over N ambient worlds there are exactly 2^N extensional states (`Model.card_allExactStates`), and crisp STE has full subset expressivity—a single property set can carve out *any* subset of the universe (`arbitrary_subset_as_single_property`)—so without structural assumptions on the constraints, no representation smaller than 2^N can be complete.

The question of this note: *which structural assumptions buy which representations?* The natural language for the question is sheaf-theoretic, because "represent locally and glue" is exactly what a sheaf condition asserts.

2 Gluing over the constraint poset

Fix $S : I \to \mathcal{P}(\Xi)$ and write, for a subset $J \subseteq I$ of the constraint index,

$$\mathrm{feas}(J) \;=\; \bigcap_{i \in J} S_i \qquad (\texttt{partialFeasibilitySet}\ S\ J),$$

the set of hypotheses consistent with the information in J. The assignment $J \mapsto \mathrm{feas}(J)$ is a presheaf on the poset $\mathcal{P}(I)$ (ordered by inclusion, mapping contravariantly): if $J \subseteq K$ then $\mathrm{feas}(K) \subseteq \mathrm{feas}(J)$, which is the information-monotonicity lemma `partialFeasibilitySet_antitone` already verified in `Ste.Basic`.

The new, machine-checked content of `Ste.Sheaf` is that this presheaf satisfies the gluing condition exactly, with no sheafification needed. The engine is the Formal Concept Analysis bridge of `Ste.HyperFrame` [5]: partial feasibility is definitionally the lower polar of the satisfaction context (`partialFeasibilitySet_eq_lowerPolar`), and polars turn unions into intersections.

Theorem 1 (`partialFeasibilitySet_iUnion`)**.** *For any family $\mathcal{J} : \kappa \to \mathcal{P}(I)$ of constraint subsets,*

$$\mathrm{feas}\Big(\bigcup_k \mathcal{J}_k\Big) \;=\; \bigcap_k \mathrm{feas}(\mathcal{J}_k).$$

Theorem 2 (`feasibilitySet_eq_iInter_cover`)**.** *If $\bigcup_k \mathcal{J}_k = I$ is any cover of the constraint index, then the global feasible set is the intersection of the local ones:*

$$\mathrm{feas}(I) \;=\; \bigcap_k \mathrm{feas}(\mathcal{J}_k).$$

In sheaf language: the functor feas sends colimits (unions) of constraint sets to limits (intersections) of feasible sets, so *local consistency data over any cover glues uniquely to the global feasible set*. On the constraint side of the Galois connection there is no obstruction whatsoever—this is a theorem, not a heuristic. Whatever makes STE hard does not live here.

3 The tractable upper bound: rectangular constraints

Where the hardness *can* live is the variable side. Take $\Xi = \prod_{v \in V} A_v$. Call a constraint *rectangular* (variable-separable, a cylinder in every coordinate) if it has the form $\prod_v P_v$ for some family of per-variable sets $P_v \subseteq A_v$—in Lean, `Set.univ.pi` P. A rectangular constraint inspects each variable independently; it couples nothing.

Theorem 3 (`rectangular_feasibilitySet`)**.** *If every constraint is rectangular, $S_i = \prod_v P_{i,v}$, then the feasible set is itself the rectangle whose v-th side is the pointwise intersection:*

$$\bigcap_i \prod_v P_{i,v} \;=\; \prod_v \Big(\bigcap_i P_{i,v}\Big).$$

This is the representation theorem for the easy case: a fully variable-separable STE problem is represented *exactly* by one subset per variable. The representation lives in $\sum_v |A_v|$ space—linear in the number of variables—never in the $\prod_v |A_v|$ product space. The 2^N wall does not exist for rectangular constraint families.

The cardinality statement is checked as well, in terms of Mathlib's extended natural cardinality encard (which handles infinite sides gracefully):

Theorem 4 (`rectangular_encard`)**.** *For a finite variable set V (`Fintype` V),*

$$\Big|\bigcap_i \prod_v P_{i,v}\Big| \;=\; \prod_{v\in V}\Big|\bigcap_i P_{i,v}\Big|,$$

as extended cardinals.

So in the separable case even the *size* of the feasible set is a product of locally-computable sizes: consistency checking, counting, and sampling all decompose per-variable. This is the STE incarnation of the trivial end of CSP tractability: a constraint network with no edges is backtrack-free [4].

4 The obstruction: coupling is not rectangular

The lower bound is a single, minimal counterexample. Let the hypothesis space be two binary variables, $\Xi = (\mathrm{Fin}\,2 \to \mathrm{Bool})$, and consider the *diagonal* (coupling, equality) constraint

$$\Delta \;=\; \{\, f \mid f(0) = f(1) \,\} \qquad (\texttt{diagonal}).$$

Theorem 5 (`diagonal_not_rectangular`)**.** *There is no family $t : \mathrm{Fin}\,2 \to \mathcal{P}(\mathrm{Bool})$ with $\Delta = \prod_v t_v$.*

The proof is the two-line contextuality argument: $(0,0) \in \Delta$ and $(1,1) \in \Delta$, so any rectangle containing Δ has $0 \in t_0$ and $1 \in t_1$, hence contains the mixed point $(0,1)$ —which violates the coupling. The local (per-variable) data of Δ is *full*: projected to either coordinate, everything is possible. The constraint is invisible locally and exists only jointly.

This is precisely the elementary form of the sheaf-theoretic obstruction of Abramsky and Brandenburger [1]: a family of perfectly consistent local sections (here: both values possible at each variable) that admits no factorization through a product—the "global section" data is not determined by, and not recoverable from, the per-variable marginals. In cohomological terms (made precise only in the outlook below), the diagonal carries a nonzero Čech gluing class relative to the cover of V by singletons.

It is also, structurally, the same countermodel this project already uses for a different purpose: `Ste.DynamicFrame.Counterexample` refutes transitivity of possible-coreference (`maySame_not_transitive`) with a three-world model in which pairwise-consistent local identifications fail to assemble into a global one. Pairwise consistency without global consistency is the one phenomenon, seen twice.

Consequences for representation: since rectangles are closed under intersection (`rectangular_feasibilitySet` again), no family of rectangular constraints can ever express Δ. Coupling constraints are therefore *exactly* the constraints that force representations beyond $\sum$-space; the 2^N wall of `card_allExactStates` is built out of them.

5 Block decomposition: $\sum$ across blocks, $\prod$ within

The module `Ste.Decomposition` proves the first positive decomposition result beyond full separability. Suppose the hypothesis space splits into two blocks, $\Xi = X \times Y$, and the constraint family is indexed by a disjoint union $I = \iota \sqcup \kappa$ where ι-constraints depend only on the X coordinate and κ-constraints only on Y (in Lean, `blockFamily` C D takes preimages under the two projections).

Theorem 6 (`feasibilitySet_blockFamily`)**.** *For any block families $C : \iota \to \mathcal{P}(X)$ and $D : \kappa \to \mathcal{P}(Y)$,*

$$\text{feas}(\text{blockFamily } C\ D) \;=\; \text{feas}(C) \times \text{feas}(D).$$

Theorem 7 (`encard_feasibilitySet_blockFamily`, `encard_feasibilitySet_blockFamily_le`)**.** *Consequently the representation cost multiplies, $|\text{feas}(\text{blockFamily } C\ D)| = |\text{feas}(C)| \cdot |\text{feas}(D)|$ as extended cardinals, and is bounded by $|X| \cdot |Y|$.*

Iterated, this is the $\sum$-across-blocks, $\prod$-within-block law: when the constraint hypergraph is disconnected, the exponential lives only inside each connected component. The question becomes: what counts as a block, and what dissolves coupling between blocks? Both answers need a machine-checked notion of *which variables a constraint actually touches.*

6 Support: the scope of a constraint

The module `Ste.Support` defines, for a constraint $T \subseteq \prod_v A_v$ and a set of variables $\sigma \subseteq V$, the predicate `HasSupport` T σ: any two assignments agreeing on σ are both in or both out of T. This is the scope of a constraint in the CSP sense [3], i.e. the hypergraph edge on which the constraint sits. Verified basic theory: supports are monotone (`HasSupport.mono`), every constraint is supported on V (`hasSupport_univ`), support $\emptyset$ holds exactly for the trivial constraints Ξ and $\emptyset$ (`hasSupport_empty_iff`) — scope-free information is no information — rectangles are supported on their non-trivial coordinates (`hasSupport_pi`), and supports are stable under intersection, so a family with common scope σ has feasible set with scope σ (`HasSupport.inter`, `hasSupport_feasibilitySet`).

The payoff is a sharpening of the coupling obstruction of Section 4 from "not rectangular" to "scope provably $\{0,1\}$":

Theorem 8 (`diagonal_support_full`, `diagonal_hasSupport_pair`)**.** *Every support of the diagonal Δ contains both variables, and $\{0,1\}$ is a support; hence the scope of Δ is exactly $\{0,1\}$. In particular Δ has no support $\emptyset$, $\{0\}$, or $\{1\}$ (`diagonal_not_hasSupport_empty`, `diagonal_not_hasSupport_zero`, `diagonal_not_hasSupport_one`).*

Where `diagonal_not_rectangular` says Δ is not a product of per-variable data, this says more: no proper subset of the variables suffices to decide membership. The coupling core is genuinely two-variable — it is an irreducible edge of the constraint hypergraph.

7 Conditioning: variable elimination, machine-checked

The module `Ste.Conditioning` mechanizes the elimination step of bucket elimination [3]: conditioning a constraint T on fixing one coordinate $v := a$,

$$\text{cond}(T, v, a) \;=\; \{\, f \mid f[v \mapsto a] \in T \,\} \qquad (\texttt{condition}).$$

Machine-checked facts:

Theorem 9 (`condition_feasibilitySet`). *Conditioning commutes with feasibility:* $\mathrm{cond}(\mathrm{feas}(S), v, a) = \mathrm{feas}(i \mapsto \mathrm{cond}(S_i, v, a))$. *Elimination may be performed constraint-by-constraint.*

Theorem 10 (`HasSupport.condition`). *Elimination shrinks scopes: if T has support σ then $\mathrm{cond}(T, v, a)$ has support $\sigma \setminus \{v\}$. The eliminated variable leaves the constraint hypergraph.*

Theorem 11 (`condition_pi`; `condition_condition_self`, `condition_condition_comm`). *The rectangular class is closed under conditioning on a feasible value (the v-th side becomes trivial), and elimination is idempotent per variable and commutes across distinct variables — the elimination order is immaterial.*

Theorem 12 (`condition_diagonal_rectangular`, `condition_diagonal_hasSupport_one`). *Conditioning the diagonal on either value of variable* 0 *yields a* rectangular *constraint of scope* $\{1\}$ *— although the diagonal itself is provably not rectangular and provably has no support* $\{1\}$. *Fixing one variable of the coupling core moves it from the intractable class to the tractable one.*

Finally, the cut-variable theorem connects elimination to the block decomposition of Section 5. Say the family S is a *block family* for the partition $\{\sigma, \sigma^{\mathsf{c}}\}$ if every constraint has support inside σ or inside σ^{c}; and say v is a *cut variable* if every constraint has support inside $\sigma \cup \{v\}$ or $\sigma^{\mathsf{c}} \cup \{v\}$ — the two sides interact only through v.

Theorem 13 (`piecewise_mem_feasibilitySet`). *The feasible set of a block family is closed under block-wise recombination: if $f, g \in \mathrm{feas}(S)$ then the splice σ.*piecewise *$f\ g$ (equal to f on σ, to g off σ) is again in* $\mathrm{feas}(S)$. *This is the pi-space form of "*$\mathrm{feas}(S)$ *is a product of two independent subproblems" (cf.* `feasibilitySet_blockFamily`*).*

Theorem 14 (`condition_cut_blocks`, `piecewise_mem_condition_feasibilitySet`). *Conditioning on a cut variable disconnects the problem: after fixing $v := a$, every conditioned constraint has support inside a single block, and the conditioned feasible set* $\mathrm{cond}(\mathrm{feas}(S), v, a)$ *is closed under block-wise recombination.*

This is variable elimination confining the exponential: eliminating a cut vertex of the constraint hypergraph splits the remaining problem into independent subproblems, each paying only its own product cost. The next section makes the cut theorem general and the cost per bag quantitative.

8 Cut elimination and treewidth: the bag invariant

The module `Ste.Treewidth` proves, over an abstract variable type V with alphabets A_v, the two accounting facts that underlie treewidth bounds [3], then the quantitative per-bag cost.

Scope accounting

Theorem 15 (`HasSupport.inter_union`; `HasSupport.iInter`). *Joins union scopes: if T has support σ and U has support τ, then $T \cap U$ has support $\sigma \cup \tau$; for a family, $\bigcap_i T_i$ has support $\bigcup_i \sigma_i$.*

Theorem 16 (`condition_inter`). *Conditioning distributes over intersection:* $\mathrm{cond}(T \cap U, v, a) = \mathrm{cond}(T, v, a) \cap \mathrm{cond}(U, v, a)$. *Elimination commutes with joining.*

With `HasSupport.condition` (elimination removes v from every scope), the scope of an intermediate table of bucket elimination is $(\bigcup_i \sigma_i) \setminus \{v\}$ — the *bag* of the step.

Cut elimination, general form

Theorem 17 (`cut_elimination`)**.** *If T, U have supports σ, τ with $\sigma \cap \tau \subseteq \{v\}$ (the constraints share only the cut variable v), then for any value a the conditioned constraints* $\mathrm{cond}(T, v, a)$, $\mathrm{cond}(U, v, a)$ *have* disjoint *supports $\sigma \setminus \{v\}$ and $\tau \setminus \{v\}$.*

Theorem 18 (`inter_nonempty_iff_of_disjoint_support`; `condition_inter_nonempty_iff`)**.** *Disjoint scopes make joint feasibility independent: for supports $\sigma \perp \tau$, $T \cap U \neq \emptyset$ iff $T \neq \emptyset$ and $U \neq \emptyset$ (splice the witnesses along σ). Consequently, if T, U share only the cut variable v, then after conditioning on v the joint problem is satisfiable iff each side is satisfiable separately. This is the pi-space form of the block product `feasibilitySet_blockFamily`: fixing a cut variable dissolves the join into independent checks.*

The bag bound

For a scope $\sigma \subseteq V$, the *table* of a constraint T is its restriction image $\mathrm{table}(\sigma, T) = \{\, f|_\sigma \mid f \in T \,\}$ (`table`) — the relation bucket elimination materializes.

Theorem 19 (`HasSupport.eq_preimage_table`)**.** *A constraint with support σ is the preimage of its σ-table: the table is a faithful representation, losing no information.*

Theorem 20 (`table_encard_le`; `table_encard_le_pow`)**.** *For finite σ and finite alphabets, the table of* any *constraint satisfies $|\mathrm{table}(\sigma, T)| \leq \prod_{v \in \sigma} |A_v|$; if moreover $|A_v| \leq k$ for all v and $|\sigma| \leq w+1$, then $|\mathrm{table}(\sigma, T)| \leq k^{w+1}$.*

Theorem 21 (`elimination_step`)**.** *The elimination step of bucket elimination—join constraints with scopes σ_i, then eliminate v—produces a constraint that (i) has support the bag $(\bigcup_i \sigma_i) \setminus \{v\}$, (ii) equals the preimage of its bag table, and (iii) has bag table of at most k^{w+1} rows whenever the bag has at most $w+1$ variables over alphabets of size at most k.*

This is the per-step space invariant of variable elimination: an elimination whose every bag stays within $w+1$ variables never materializes more than k^{w+1} rows at a step — the induced-width cost a^{w^*+1} [3]. *Not yet proven* (outlook): the multi-step fold over an elimination *order* (iterating `elimination_step` along an ordering and carrying the width invariant through the recursion), a tree-decomposition datatype with its junction-tree representation theorem, and the `encard` factorization of the conditioned feasible set through restriction maps (the quantitative analogue of `encard_feasibilitySet_blockFamily`).

9 Outlook: the cohomological program (not proven)

Everything in this section is conjecture or programme. None of it is machine-checked, and none of it should be cited as a result of this project.

Conjecture (outlook) (Feasibility as a sheaf on a site over variables)**.** *Section 2 proved gluing over the* constraint *poset. The substantive open direction is the* variable *side: assign to each subset $W \subseteq V$ of variables the set of partial assignments on W consistent with all constraints whose support lies in W. We conjecture this is a presheaf on a Grothendieck site over $\mathcal{P}(V)$ whose sheaf condition, relative to a cover of V by constraint supports, holds precisely for the tractable fragments; the diagonal example shows the presheaf is not a sheaf for the singleton cover in general.*

Conjecture (outlook) (k-consistency is higher gluing)**.** *Local (k-)consistency in the CSP sense [3, 4]—every consistent assignment on $k-1$ variables extends to any k-th—should be exactly the statement that sections glue along the cover of V by k-element subsets. Freuder's theorem (strong k-consistency plus width $< k$ implies backtrack-free search) would then become: on covers refined enough for the treewidth, the presheaf* is *a sheaf, and a global section is constructed greedily.*

Conjecture (outlook) ($\check{H}^1$ measures intractability)**.** *Following Abramsky and Brandenburger [1], the obstruction to a family of compatible local sections extending to a global one should be captured by a Čech cohomology class in $\check{H}^1$ of the support cover with coefficients in the (relative) presheaf of partial solutions; the class vanishes iff gluing succeeds. The conjectured quantitative version: the difficulty of an STE instance—the size blow-up of its smallest exact representation—is controlled by the size of this obstruction, with rectangular families the $\check{H}^1 = 0$ case of Section 3 and the diagonal of Section 4 the minimal nonvanishing class.*

Conjecture (outlook) (Tractability dichotomy)**.** *Bounded treewidth of the constraint (co)occurrence structure—or, conjecturally equivalent in this setting, bounded obstruction in the sense above—yields a polynomial-size glued representation (junction-tree style: rectangles over bags, glued along separators), and unbounded coupling reproduces the 2^N wall. This would be the STE form of the classical CSP dichotomy between bounded-width tractability and general intractability [3].*

What stands *proven* against this programme: the fully separable case is exactly linear (`rectangular_feasibili` `rectangular_encard`); one coupling suffices to leave it, and its scope is provably irreducible (`diagonal_not_rectangular`, `diagonal_support_full`); disconnected blocks multiply (`feasibilitySet_blockFa` `encard_feasibilitySet_blockFamily`); eliminating a variable removes it from every scope (`HasSupport.conditio` eliminating a *cut* variable disconnects the residual problem into independent blocks with disjoint scopes (`piecewise_mem_condition_feasibilitySet`, `cut_elimination`) whose joint feasibility factors into independent checks (`condition_inter_nonempty_iff`); and every single elimination step is confined to its bag with table cost at most $\prod_{v\in\mathrm{bag}} |A_v| \le k^{w+1}$ (`elimination_step`, `table_encard_le`)—the width-1 case of the conjectured dichotomy, plus its per-step space bound. The constraint-side gluing (`feasibilitySet_eq_iInter_cover`) guarantees the difficulty is genuinely about variables, not about how information is indexed. Open in this direction: the multi-step fold over an elimination ordering, the quantitative `encard` factorization of the conditioned feasible set over a partition, junction trees, and the full treewidth dichotomy.

References

[1] Samson Abramsky and Adam Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13:113036, 2011. doi: 10.1088/1367-2630/13/11/113036. arXiv:1102.0264.

[2] Patrick L. Combettes. The foundations of set theoretic estimation. *Proceedings of the IEEE*, 81 (2):182–208, 1993. doi: 10.1109/5.214546. Retrieved from the author's page, `https://pcombet.math.ncsu.edu/proc.pdf`.

[3] Rina Dechter. *Constraint Processing.* Morgan Kaufmann, San Francisco, 2003.

[4] Eugene C. Freuder. A sufficient condition for backtrack-free search. *Journal of the ACM*, 29 (1):24–32, 1982. doi: 10.1145/322290.322292.

[5] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations.* Springer-Verlag, Berlin, Heidelberg, 1999. Translated by Cornelia Franzke.